YACHAY TECH UNIVERSITY

//01

Decrease and Conquer

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Decrease-and-Conquer

1. Reduce problem instance to smaller instance of the same problem
2. Solve smaller instance
3. Extend solution of smaller instance to obtain solution to original instance

- Can be implemented either top-down or bottom-up
- Also referred to as inductive or incremental approach

3 Types of Decrease and Conquer

Decrease by a constant (usually by 1):

- insertion sort
- topological sorting
- algorithms for generating permutations, subsets

Decrease by a constant factor (usually by half)

- binary search and bisection method
- exponentiation by squaring
- multiplication à la russe

Variable-size decrease

- Euclid's algorithm
- selection by partition
- Nim-like games

What's the difference?

Consider the problem of exponentiation: Compute a^n

- Brute Force: $a^n = a \times a \times \ldots \times a$
- Decrease by one: $a^n = a^{n-1} \times a$
- Decrease by constant factor: $a^n = (a^{n/2})^2$

Insertion Sort

To sort array A[0..n-1], sort A[0..n-2] recursively and then insert A[n-1] in its proper place among the sorted A[0..n-2]

- Usually implemented bottom-up (nonrecursively)

Example: Sort 6, 4, 1, 8, 5

6	| 4	1	8	5
4	6	| 1	8	5
1	4	6	| 8	5
1	4	6	8	| 5
1	4	5	6	8

Pseudocode of Insertion Sort

```
ALGORITHM  InsertionSort(A[0..n-1])
  //Sorts a given array by insertion sort
  //Input: An array A[0..n-1] of n orderable elements
  //Output: Array A[0..n-1] sorted in nondecreasing order

  for i  ←  to n-1 do
    v ← A[i]
    j ← i-1
    while j≥0 and A[j]>v do
      A[j+1] ←  A[j]
      j ← j -1
   A[j+1] ← v
```

Analysis of Insertion Sort

- Time efficiency
  - $C_{worst}(n) = n(n-1)/2 \in \Theta(n^2)$
  - $C_{avg}(n) \approx n^2/4 \in \Theta(n^2)$
  - $C_{best}(n) = n - 1 \in \Theta(n)$ (also fast on almost sorted arrays)
- Space efficiency: in-place
- Stability: yes
- Best elementary sorting algorithm overall
- Binary insertion sort

Decrease by a Constant

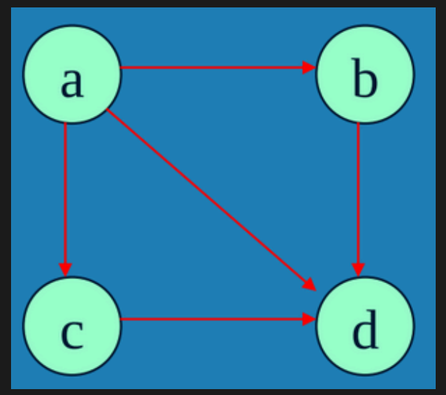
Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

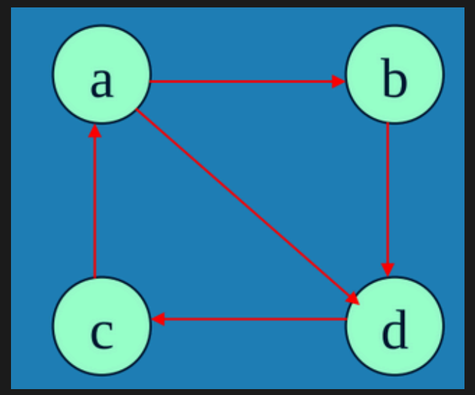
DAGs and Topological Sorting

DAG: a directed acyclic graph, i.e. a directed graph with no (directed) cycles

aa

a DAG





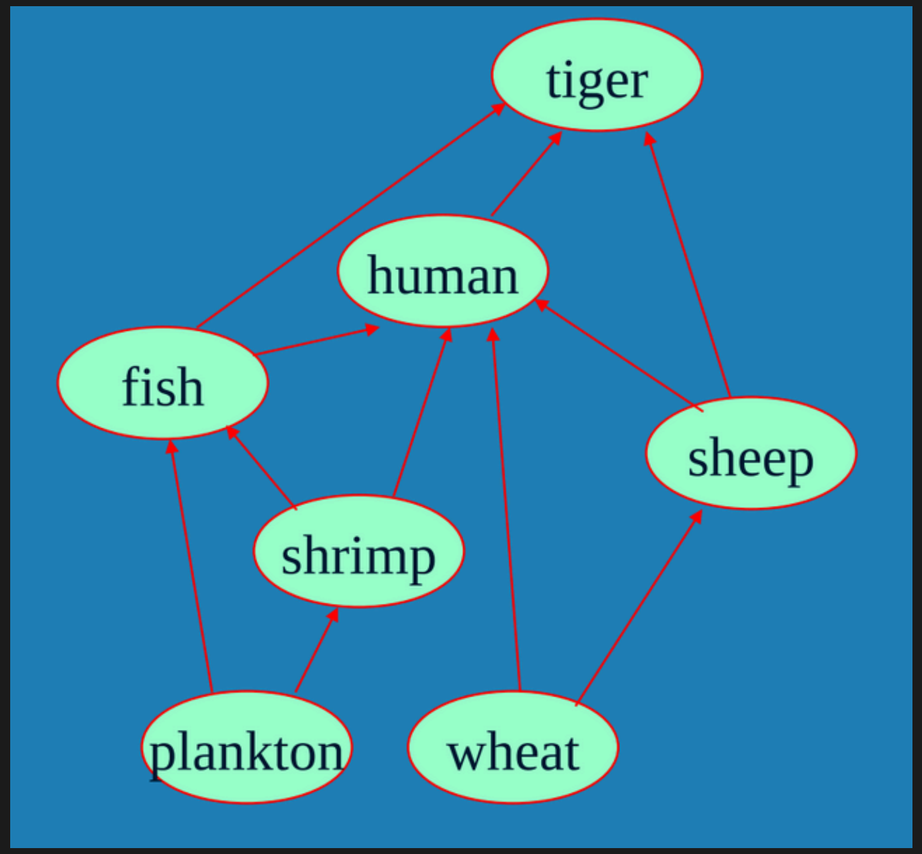
not a DAG

Arise in modeling many problems that involve prerequisite constraints (construction projects, document version control)

Nodes of a dag can be linearly ordered so that for every edge its starting node is listed before its ending node (topological sorting). Being a DAG is also a necessary condition for topological sorting to be possible.

Topological Sorting Example

Order the following items in a food chain

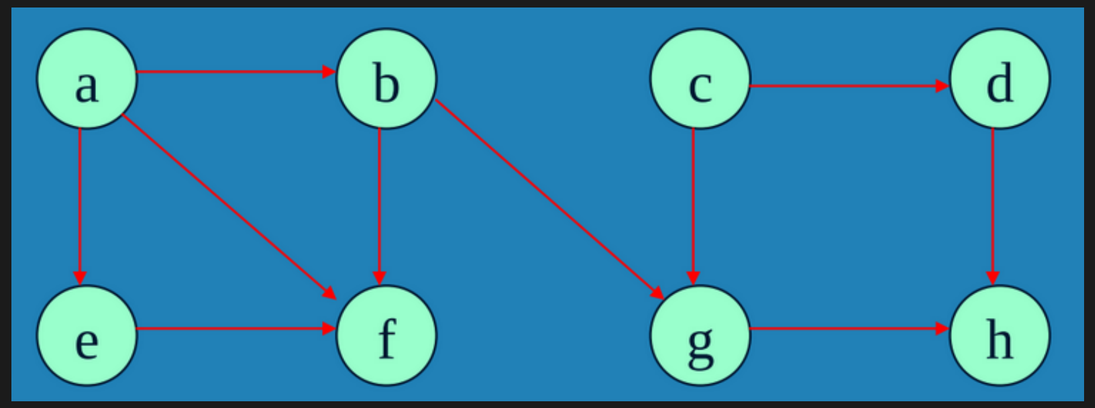


DFS-based Algorithm

<u>DFS-based algorithm for topological sorting</u>

- Perform DFS traversal, noting the order nodes are popped off the traversal stack
- Reverse order solves topological sorting problem
- Back edges encountered?→ NOT a DAG!

Example:

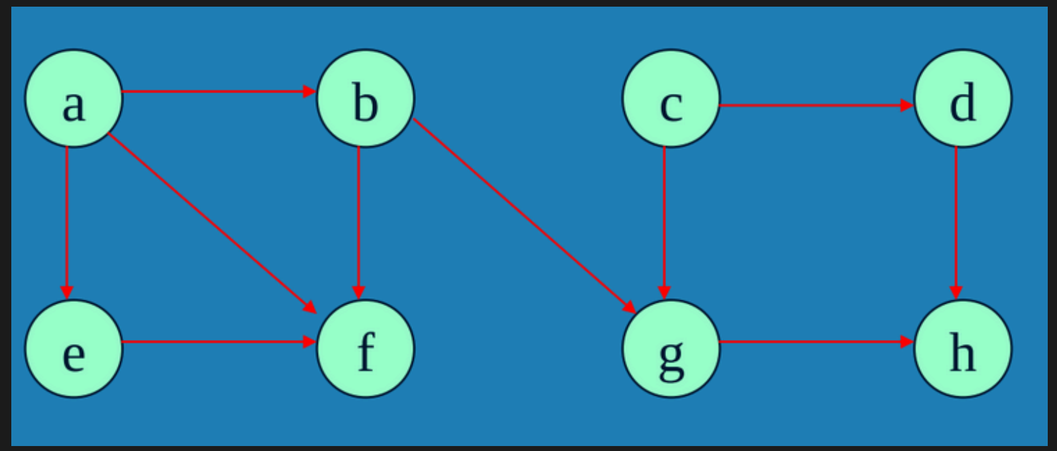


Efficiency:

Source Removal Algorithm

Repeatedly identify and remove a source (a node with no incoming edges) and all the edges incident to it until either no node is left (problem is solved) or there is no source among remaining nodes (not a DAG).

Example:



Efficiency: same as efficiency of the DFS-based algorithm

Generating Permutations

<u>Minimal-change</u> decrease-by-one algorithm

If n = 1 return 1; otherwise, generate recursively the list of all permutations of 12…n-1 and then insert n into each of those permutations by starting with inserting n into 12…n-1 by moving right to left and then switching direction for each new permutation.

Example: n=3

start				1			
insert 2 into 1 right to left		12				21	
insert 3 into 12 right to left	123	132	312				
insert 3 into 21 left to right					321	231	213

Other permutation generating algorithms

- Johnson-Trotter
- Lexicographic-order algorithm
- Heap’s algorithm

Generating Subsets

Binary Reflected Gray Code: a minimal-change algorithm for generating 2^n bit strings corresponding to all the subsets of an n-element set where $n > 0$

```
ALGORITHM BRGC(n)
  //Generates recursively the binary reflected Gray code of order n
  //Input: A positive integer n
  //Output: A list of all bit strings of length n composing the Gray code

  if n = 1 make list L containing bit strings 0 and 1 in this order
  else generate list L1 of bit strings of size n - 1 by calling BRGC(n - 1)
    copy list L1 to list L2 in reversed order
    add 0 in front of each bit string in list L1
    add 1 in front of each bit string in list L2
    append L2 to L1 to get list L
  return L
```

Q & A

FOR QUESTIONS OR COMMENTS

EMAIL ADDRESS

ipineda@yachaytech.edu.ec